

Recording

Date : 08/06/2026

Problem Specification : Create an array from user input, search for a number and return its index, then sort the array in ascending order and display the output.

Assumption :

- The array size entered by the user is greater than 0.
- If the searched number appears multiple times in the array, the program returns the index of its first occurrence.
- All array elements are integers.
- The user enters valid integer values.

Limitation :

- The program only handles integer arrays.
- Linear search is used, which may be inefficient for very large arrays.
- Bubble sort is used, which has a higher time complexity compared to -advanced sorting algorithms.

Input :

- Array size (n)
- n integer values for the array
- Search number (num)

Processing :

- The program reads the array elements, searches for a specified number using Linear Search, displays its index if found, sorts the array in ascending order using Bubble Sort, and displays the sorted array.

Output :

- Unsorted Array
- Index of the searched number (or not found message)
- Sorted array in ascending order

Algorithm :

- Step 1:** Start the program.
- Step 2:** Read the array size and store it in **n**.
- Step 3:** Create an integer array **numArray[n]**.
- Step 4:** Read and store **n** elements in **numArray**.
- Step 5:** Call **printArray(numArray)** to display the unsorted array.

Step 6: Read the search number and store it in **num**.

Step 7: Call **elementSearch(numArray, num)** and store the returned value in **index**.

Step 8: Display the value of **index**.

Step 9: Call **sortingArray(numArray)** to sort the array in ascending order using Bubble Sort and assign int array sortNumArray

sortNumArray $\leftarrow$ sortingArray(numArray)

Step 10: Call **printArray(sortNumArray)** to display the sorted array.

Step 11: End the program.

Method: elementSearch(numArray, num)

Step 1: Initialize **i** $\leftarrow$ 0.

Step 2: Repeat while **i** < **numArray.length**:

- If **numArray[i] == num**, return **i**.
- Otherwise, increment **i** $\leftarrow$ **i + 1**.

Step 3: Return **-1** if the number is not found.

Method: sortingArray(numArray)

Step 1: Let **n** $\leftarrow$ **numArray.length**.

Step 2: Initialize **i** $\leftarrow$ 0.

Step 3: Repeat while **i** < **n - 1**:

- Initialize **j** $\leftarrow$ 0.
- Repeat while **j** < **n - i - 1**:
 - If **numArray[j] > numArray[j + 1]**:
 - temp** $\leftarrow$ **numArray[j]**
 - numArray[j]** $\leftarrow$ **numArray[j + 1]**
 - numArray[j + 1]** $\leftarrow$ **temp**
 - Increment **j** $\leftarrow$ **j + 1**.
- Increment **i** $\leftarrow$ **i + 1**.

Step 4: End method.

Method: printArray(numArray)

Step 1: Traverse each element of **numArray**.

Step 2: Display each element.

Step 3: End method.

Programme listing : **Programme file attached**

Test data and expected output :

1. Test data: n= 5
 Array Elements:10 5 8 2 7
 Search Number:num=8

 Expected output: Unsorted Array:10 5 8 2 7
 Number found at index 2
 Sorted Array: 2 5 7 8 10

2. Test data: N = 6
 Array Elements: 5 6 4 3 2 4
 Search Number:num=8

 Expected output: Unsorted Array: : 5 6 4 3 2 4
 Number found at index -1
 Sorted Array: 2 3 4 4 5 6

3. Test data: n = 6
 Array Elements: 5 6 4 3 2 4
 Search Number:num=4

 Expected output: Unsorted Array: : 5 6 4 3 2 4
 Number found at index 2
 Sorted Array: 2 3 4 4 5 6

Output obtained for test data :

1. Test data: n= 5
Array Elements:10 5 8 2 7
Search Number:num=8

Expected output: Unsorted Array:10 5 8 2 7

Number found at index 2

Sorted Array: 2 5 7 8 10

2. Test data: n = 6
Array Elements: 5 6 4 3 2 4
Search Number:num=8

Expected output: Unsorted Array: : 5 6 4 3 2 4

Number found at index -1

Sorted Array: 2 3 4 4 5 6

3. Test data: n = 6
Array Elements: 5 6 4 3 2 4
Search Number:num=4

Expected output: Unsorted Array: 5 6 4 3 2 4

Number found at index 2

Sorted Array: 2 3 4 4 5 6

Analysis :**Linear Search**

- Best Case: $\Theta(1)$
- Worst Case: $\Theta(n)$
- Average Case: $\Theta(n)$

Bubble Sort

- Best Case: $\Theta(n^2)$
- Worst Case: $\Theta(n^2)$
- Average Case: $\Theta(n^2)$

Conclusion

: This program successfully performs searching and sorting operations on an integer array. Linear Search is used to locate a specified element and return its index, while Bubble Sort is used to arrange the array elements in ascending order. The program produces the expected output for all test cases.

Discussion :

The running time of the Linear Search algorithm depends on the position of the searched element. In the worst case, every element must be examined, resulting in $\Theta(n)$ complexity.

Bubble Sort repeatedly compares and swaps adjacent elements until the array becomes sorted. Its running time is $\Theta(n^2)$, making it suitable for small datasets but inefficient for large arrays.

For improved performance, algorithms such as Binary Search (after sorting) and Quick Sort or Merge Sort could be used.